
FiX: Introducing Fine-grained Forget Gate into Softmax Attention

Runzhong Li^{1,2} Renjie Liu^{1†} Qing Li² Bo Tang¹

Causal softmax attention is the algorithmic foundation of modern large language models. Inspired by linear attention, recent work has sought to enhance it by incorporating forget gates. However, these efforts, such as FoX, have been limited to coarse, scalar gates. While fine-grained, element-wise gates are shown to be more effective than scalar ones in linear attention, their direct integration into softmax attention is non-trivial due to algebraic constraints. In this work, we introduce *Fine-grained Forgetting Transformer (FiX)*, a novel architecture that successfully enables element-wise forget gates in softmax attention. Our core insight is that the softmax denominator becomes mathematically redundant under a subsequent RMSNorm layer, allowing us to reformulate the forgetting mechanism as a direct element-wise multiplication on the value vectors. This formulation makes FiX the first positional encoding applied to value-output (VO) pairs, designed to be complementary to existing query-key (QK) encodings like RoPE. We systematically address implementation challenges including numerical precision, computational efficiency, and inference memory consumption. Extensive experiments show *FiX* achieves lower training loss and superior performance on both short-text common sense benchmarks and long-context tasks, opening a new path for building more powerful transformers. Our code is available at <https://github.com/AlayaDB-AI/FiX>.

1. Introduction

Despite the surge of research into efficient alternatives like recurrent sequence models (Yang et al., 2025b; Zhang et al., 2025b) to mitigate the quadratic complexity of standard attention, they frequently fall short in long-context scenarios due to the inherent constraints of their fixed-size recurrent states (Wen et al., 2025). Consequently, causal softmax attention (Vaswani et al., 2017) remains the unrivaled algorithmic foundation and an essential component for modern

large language models (LLMs) (DeepSeek-AI et al., 2025; Yang et al., 2025a; Team et al., 2025). To further enhance its capabilities, recent work such as Forgetting Transformers (FoX) (Lin et al., 2025) has integrated forget gate mechanisms (Hochreiter & Schmidhuber, 1997; Sun et al., 2023) from recurrent sequence models into the softmax attention, enabling models to selectively discard historical information based on context. However, these efforts have been limited to coarse, scalar gates that treat all feature dimensions with a uniform decay rate. This lack of fine-grained, element-wise control is a significant limitation, as such mechanisms have been shown to effectively improve modern linear attention models, e.g., Gated Linear Attention (Yang et al., 2024) and Kimi Delta Attention (Zhang et al., 2025b).

However, translating this success from linear attention to softmax attention is non-trivial, since directly extending FoX to support fine-grained gating is infeasible. Specifically, in FoX, the forgetting mechanism is applied as a bias term $D_{i,j}$ added to the attention logit $q_i k_j^\top$. Since the dot product of query and key vectors yields a scalar, it is algebraically impossible to add a vector-valued bias (namely a multi-dimensional forget gate) to this scalar logit.

To enable vectorized forget gates in softmax attention, we analyze the behavior of softmax attention when followed by a Root Mean Square Layer Normalization (RMSNorm) (Zhang & Sennrich, 2019), a standard component in modern LLM architectures (Yang et al., 2025a). Interestingly, we find that under RMSNorm, the normalization denominator of the softmax operation becomes redundant. This insight suggests that the mathematical effect of a forget gate within the softmax mechanism is functionally equivalent to applying cumulative forget gates to the value vectors.

Under this observation, we propose the *Fine-grained Forgetting Transformer (FiX)*. By decoupling the forgetting mechanism from the attention scores, *FiX* achieves the fine-grained control necessary for handling complex long-context dependencies without sacrificing the foundational strengths of the transformer architecture. Moreover, this fine-grained forget gate also serves as V-O position embeddings, making it complementary with existing Q-K positional encoding like RoPE (Su et al., 2024). To our understanding, *FiX* is the first positional encoding on the V-O vectors.

For hardware-efficient training, we implement *Flash Fine-*

[†]Work done during internship at AlayaDB. ¹Department of Computer Science and Engineering, Southern University of Science and Technology, Shenzhen, China ²Department of Computing, The Hong Kong Polytechnic University, Hong Kong, China. Correspondence to: Bo Tang <tangb3@sustech.edu.cn>, Qing Li <csqli@comp.polyu.edu.hk>.

grained Attention, a FlashAttention-like (Dao, 2024) algorithm designed for *FiX*. This algorithm combines the rescaling trick from Section 3.2.3 with the tiling strategy of FlashAttention to balance numerical stability and efficiency. To improve the numerical stability of RMSNorm in our setting ($\epsilon = 10^{-30}$ in O-Norm), we fuse the *FiX*, O-Norm, and the subsequent linear layer into a single operator. For inference, we introduce a memory-saving cache to balance efficiency and memory usage, which integrates seamlessly with PagedAttention (Kwon et al., 2023). On moderate-scale language modeling with 760M-parameter transformers, our experiments show that *FiX* outperforms baselines such as RoPE and FoX on various tasks. Moreover, *FiX* can work with RoPE to get further improvements.

2. Background and Motivation

This section briefly reviews several key attention variants, identifies the main obstacle for applying a fine-grained forget gate to softmax attention, and presents the key observation that motivates *FiX*.

2.1. Softmax Attention and Linear Attention

In causal sequence modeling, each time step uses only historical information. Within this framework, an attention layer takes a sequence of input vectors $\mathbf{x}_i$ with $i \in \{1, \dots, N\}$, where N is the sequence length and i is the time step. All other sequences are projected from $\mathbf{x}_i$, for example $\mathbf{q}_i = \mathbf{x}_i \mathbf{W}_q$, $\mathbf{k}_i = \mathbf{x}_i \mathbf{W}_k$, and $\mathbf{v}_i = \mathbf{x}_i \mathbf{W}_v$. The layer then produces a sequence of outputs $\mathbf{o}_i$.

Standard causal softmax attention is defined as:

$$\mathbf{o}_i = \frac{\sum_{j=1}^i \exp(\mathbf{q}_i \mathbf{k}_j^T) \mathbf{v}_j}{\sum_{j=1}^i \exp(\mathbf{q}_i \mathbf{k}_j^T)}. \quad (1)$$

We call this the *parallel form* following (Yang et al., 2024) as each $\mathbf{o}_i$ can be computed independently. Then, replacing the term $\exp(\mathbf{q} \mathbf{k}^T)$ with a kernel $\phi(\mathbf{q}) \phi(\mathbf{k})^T$, where $\phi(\cdot)$ is a non-negative feature map, yields the parallel form of a linear attention (Katharopoulos et al., 2020):

$$\begin{aligned} \mathbf{o}_i &= \frac{\sum_{j=1}^i \phi(\mathbf{q}_i) \phi(\mathbf{k}_j)^T \mathbf{v}_j}{\sum_{j=1}^i \phi(\mathbf{q}_i) \phi(\mathbf{k}_j)^T} \\ &= \frac{\phi(\mathbf{q}_i) \sum_{j=1}^i \phi(\mathbf{k}_j)^T \mathbf{v}_j}{\phi(\mathbf{q}_i) \sum_{j=1}^i \phi(\mathbf{k}_j)^T}. \end{aligned} \quad (2)$$

This expression can be written in *recurrent form* equivalently as an RNN, where $\mathbf{S}_i$ and $\mathbf{z}_i$ are fixed-size RNN states:

$$\begin{aligned} \mathbf{S}_i &= \mathbf{S}_{i-1} + \phi(\mathbf{k}_{i-1})^T \mathbf{v}_i, \\ \mathbf{z}_i &= \mathbf{z}_{i-1} + \phi(\mathbf{k}_i), \\ \mathbf{o}_i &= \frac{\mathbf{q}_i \mathbf{S}_i}{\mathbf{q}_i \mathbf{z}_i}. \end{aligned} \quad (3)$$

2.2. Forget Gate and Forgetting Attention (FoX)

The forget gate is a long-established component of RNNs, e.g., LSTM (Hochreiter & Schmidhuber, 1997). Here we define a data-dependent forget gate as a scalar $f_i = \text{sigmoid}(\mathbf{x}_i \mathbf{w}_f^T + b)$. Applying this gate to the recurrent form of the above linear attention yields:

$$\begin{aligned} \mathbf{S}_i &= f_i \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)^T \mathbf{v}_i, \\ \mathbf{z}_i &= f_i \mathbf{z}_{i-1} + \phi(\mathbf{k}_i). \end{aligned} \quad (4)$$

This is equivalent to the following parallel form:

$$\mathbf{o}_i = \frac{\sum_{j=1}^i \left(\prod_{t=j+1}^i f_t \right) \phi(\mathbf{q}_i) \phi(\mathbf{k}_j)^T \mathbf{v}_j}{\sum_{j=1}^i \left(\prod_{t=j+1}^i f_t \right) \phi(\mathbf{q}_i) \phi(\mathbf{k}_j)^T}. \quad (5)$$

Substituting the feature-map kernel $\phi(\mathbf{q}) \phi(\mathbf{k})^T$ with the original exponential kernel $\exp(\mathbf{q} \mathbf{k}^T)$ used in softmax attention gives the forgetting attention mechanism in FoX:

$$\mathbf{o}_i = \frac{\sum_{j=1}^i \left(\prod_{t=j+1}^i f_t \right) \exp(\mathbf{q}_i \mathbf{k}_j^T) \mathbf{v}_j}{\sum_{j=1}^i \left(\prod_{t=j+1}^i f_t \right) \exp(\mathbf{q}_i \mathbf{k}_j^T)}. \quad (6)$$

2.3. Fine-grained Forget Gate

Many linear-attention variants employ a fine-grained forget gate, which has been shown to outperform a scalar gate. This data-dependent fine-grained forget gate can be defined as the vector $\mathbf{f}_i = \text{sigmoid}(\mathbf{x}_i \mathbf{W}_f + \mathbf{b})$. Applying this gate to the recurrent form of the above linear attention gives:

$$\begin{aligned} \mathbf{S}_i &= \text{diag}(\mathbf{f}_i) \mathbf{S}_{i-1} + \phi(\mathbf{k}_i)^T \mathbf{v}_i, \\ \mathbf{z}_i &= \mathbf{f}_i \circ \mathbf{z}_{i-1} + \phi(\mathbf{k}_i), \end{aligned} \quad (7)$$

where $\circ$ denotes element-wise multiplication. Again, we can transform this recurrent form to the parallel form:

$$\mathbf{o}_i = \frac{\sum_{j=1}^i \phi(\mathbf{q}_i) \left(\prod_{t=j+1}^i \text{diag}(\mathbf{f}_t) \right) \phi(\mathbf{k}_j)^T \mathbf{v}_j}{\sum_{j=1}^i \phi(\mathbf{q}_i) \left(\prod_{t=j+1}^i \text{diag}(\mathbf{f}_t) \right) \phi(\mathbf{k}_j)^T}. \quad (8)$$

However, because now the kernel $\phi(\mathbf{q}) \phi(\mathbf{k})^T$ no longer appears explicitly, we cannot directly transfer this fine-grained forget gate to softmax attention using the same substitution that derives Equation 6 from Equation 5.

2.4. Motivation: Equivalence of QK-FoX and VO-FoX

FoX, which uses scalar forgetting gates in softmax attention, can be viewed as a type of positional encoding acting on $\mathbf{q}$ and $\mathbf{k}$. Defining $\gamma_i = \prod_{t=1}^i f_t$, $\gamma_{j \rightarrow i} = \prod_{t=j+1}^i f_t = \frac{\gamma_i}{\gamma_j}$,

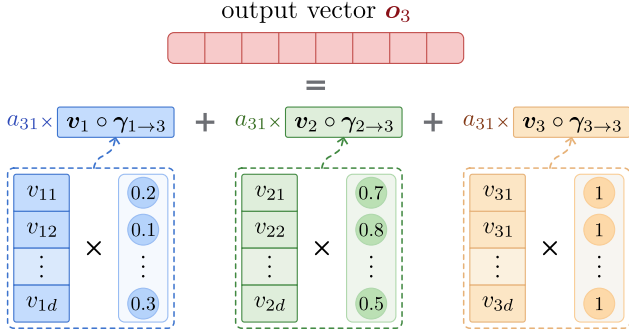


Figure 1. A conceptual illustration of FiX.

and $D_{ij} = \log \gamma_i - \log \gamma_j$, we can rewrite Equation 6 as:

$$\begin{aligned} o_i &= \frac{\sum_{j=1}^i \frac{\gamma_i}{\gamma_j} \exp(q_i k_j^T) v_j}{\sum_{j=1}^i \frac{\gamma_i}{\gamma_j} \exp(q_i k_j^T)}, \\ &= \frac{\sum_{j=1}^i \exp(q_i k_j^T + D_{ij}) v_j}{\sum_{j=1}^i \exp(q_i k_j^T + D_{ij})}. \end{aligned} \quad (9)$$

We denote this form as the QK-decay version of FoX (QK-FoX). If the decay is omitted from the denominator, Equation 10 becomes a positional encoding on v and o :

$$\begin{aligned} o'_i &= \frac{\sum_{j=1}^i \frac{\gamma_i}{\gamma_j} \exp(q_i k_j^T) v_j}{\sum_{j=1}^i \exp(q_i k_j^T)}, \\ \frac{o'_i}{\gamma_i} &= \frac{\sum_{j=1}^i \exp(q_i k_j^T) \frac{v_j}{\gamma_j}}{\sum_{j=1}^i \exp(q_i k_j^T)}. \end{aligned} \quad (10)$$

We denote this form as the VO-decay version of FoX (VO-FoX). Now we consider how VO-FoX differs from QK-FoX (standard FoX) under RMSNorm (Zhang & Sennrich, 2019), a standard component in modern LLMs defined as:

$$\text{RMSNorm}(x; w)_i = \frac{x_i w_i}{\sqrt{\frac{\sum_{j=1}^n x_j^2}{n} + \epsilon}}. \quad (11)$$

Here n is the dimension of the input vector x and ϵ is a small constant. In particular, when $\epsilon = 0$, the results of o and o' in Equation 9 and 10 are the same after RMSNorm:

$$\text{RMSNorm}(o'_i) = \text{RMSNorm}(o_i). \quad (12)$$

As the attention output is always RMS-normalized before subsequent operations, this equivalence indicates that the VO-decay version of FoX is effectively the same as standard FoX under RMSNorm. Consequently, this observation motivates us to introduce a fine-grained forget gate into softmax attention via v and o , rather than through q and k .

3. Fine-grained Forgetting Transformer

Inspired by the key insight discussed in Section 2.4, we propose *Fine-grained Forgetting Transformer (FiX)*. The core

of FiX is a causal softmax attention with fine-grained forget gates applied on v and o , which is named *Fine-grained Forgetting Attention*.

At each time step the fine-grained forget gate is a vector f_i in which each element $\in (0, 1)$. We will discuss details about the specific generation of the forget gates in Section 3.1. We define the cumulative fine-grained forget gate $\gamma_i = \prod_{j=1}^i f_j$, where the multiplication is element-wise multiplication. Then the output of the *Fine-grained Forgetting Attention* can be formulated as:

$$\begin{aligned} o_i &= \frac{\sum_{j=1}^i \exp(q_i k_j^T) (v_j \circ \prod_{t=j+1}^i f_t)}{\sum_{j=1}^i \exp(q_i k_j^T)} \\ &= \frac{\sum_{j=1}^i \exp(q_i k_j^T) \frac{v_j}{\gamma_j}}{\sum_{j=1}^i \exp(q_i k_j^T)} \circ \gamma_i. \end{aligned} \quad (13)$$

Figure 1 shows how the output o_3 is computed: each value vector v_j is first gated element-wise by the cumulative forget gate $\prod_{t=j+1}^3 f_t$, then weighted by the attention score $\exp(q_3 k_j^T)$ and summed. When $j = 3$, the cumulative forget gate $\prod_{t=4}^3 f_t$ is an empty product equal to a vector of ones, so v_3 is not changed.

3.1. Fine-grained Forget Gate in FiX

The simplest method to generate a fine-grained forget gate is to apply a sigmoid function to a linear projection, i.e., $f_i = \text{sigmoid}(x_i W_f + b_f)$. However, this approach has two main drawbacks: the linear projection introduces an excessive number of extra parameters, and the sigmoid function has a limited numerical range. To address these issues, we first employ a low-rank projection to reduce the number of parameters. Second, to achieve a wider dynamic range, we adopt an activation function adopted by Mamba (Gu & Dao, 2024). Combining these two improvements, our fine-grained forget gate is formulated as:

$$\begin{aligned} x_i^{(f)} &= \text{RMSNorm}(x_i W_f^\downarrow) W_f^\uparrow + b_f, \\ f_{i,j} &= \text{sigmoid}^a(-x_{i,j}^{(f)}), \end{aligned} \quad (14)$$

where $W_f^\downarrow$, $W_f^\uparrow$, b_f and $\log a$ are learnable parameters. The presence of the negative sign ($-$) is for the convenience of using the softplus ($\cdot$) function to calculate $\log f$.

This method works for most layers except for the first layer. This is because it is difficult to learn a projection layer of fine-grained forget gate for the discrete embedding input in the first layer. In practice, the L2 norm of the forget gate projection's gradient in the first layer is not bounded, causing huge unstability in the training process. Thus, for the first layer we learn an embedding module for the fine-grained forget gate instead of a projection layer. The forget

gate of the first layer is generated by the following equation:

$$\mathbf{f}_i = \text{sigmoid}(-\mathbf{E}_f[i_t]), \quad (15)$$

where $\mathbf{E}_f$ is the learnable embedding module for the forget gate and i_t is the token id (an integer) of the token in time step t . Note that though this extra embedding introduces more parameters, they cause no computation and can be easily offloaded to CPU memory during inference time.

RMSNorm configuration. We highlight that the ϵ used in the RMSNorm layer followed by the attention output in *FiX* should be much smaller than that in other models, considering that Equation 12 only holds when $\epsilon = 0$. In practice, most open-source SOTA LLMs (Grattafiori et al., 2024; DeepSeek-AI et al., 2025; Team et al., 2025) usually set ϵ to 10^{-5} or 10^{-6} in order to prevent division by zero and keep numerical stability. Instead of common settings, we use $\epsilon = 10^{-30}$ to satisfy the requirement of *FiX*. Since ϵ is much smaller than usual, it causes more numerical instability. As such, we fuse the RMSNorm into the attention implementation to normalize the output of attention using float32 format to keep higher precision, which is discussed in Section 4.2.

3.2. Numerical Stability

On modern GPUs, efficient training relies on Tensor Cores, specialized hardware that exclusively accelerates matrix multiplication. The computation of $\mathbf{o}$ in *FiX* can be expressed in two forms. The matrix form is suitable for Tensor Cores but is numerically unstable due to division by very small values. Conversely, the parallel form is numerically stable but cannot be computed using matrix multiplication. In this section, we first analyze the numerical instability of the matrix form and the computational limitations of the parallel form. We then introduce a rescaling trick to derive a formulation that is both numerically stable and can be computed efficiently as a matrix multiplication.

3.2.1. MATRIX FORM

We can rewrite Equation 13 in the matrix form as follows:

$$\mathbf{O} = \left(\text{Softmax}(\mathbf{Q}\mathbf{K}^T + \mathbf{M}) \left(\frac{\mathbf{V}}{\Gamma} \right) \right) \circ \Gamma, \quad (16)$$

where $\mathbf{Q}, \mathbf{K} \in \mathbb{R}^{s \times d_k}$, $\mathbf{V}, \mathbf{O}, \Gamma \in \mathbb{R}^{s \times d_v}$, d_k is the dimension of key and query, d_v is the dimension of value and output, and s is the sequence length. Each row of these matrices corresponds to a vector at that time step, e.g., the i -th row of $\mathbf{Q}$ stands for $\mathbf{q}_i$. Especially, Γ can be computed by a cumulative product of $\mathbf{F}$ along the sequence dimension, where the i -th row of $\mathbf{F}$ is $\mathbf{f}_i$. The $\mathbf{M} \in \mathbb{R}^{s \times s}$ is the causal mask matrix, whose elements are 0 in the lower triangle and $-\infty$ in the strict upper triangle.

If we define $\mathbf{V}' = \frac{\mathbf{V}}{\Gamma}$ and $\mathbf{O}' = \frac{\mathbf{O}}{\Gamma}$, where the division is element-wise, we can compute $\mathbf{O}$ in the following steps:

$$\begin{aligned} \mathbf{V}' &= \frac{\mathbf{V}}{\Gamma}, \\ \mathbf{O}' &= \text{Softmax}(\mathbf{Q}\mathbf{K}^T + \mathbf{M}) \mathbf{V}', \\ \mathbf{O} &= \mathbf{O}' \circ \Gamma. \end{aligned} \quad (17)$$

The second line of Equation 17 is the standard causal softmax attention. In this matrix form, the entire sequence is processed at once using a single, transformed value matrix $\mathbf{V}'$. This allows the use of a single, large matrix multiplication $\mathbf{Q}\mathbf{K}^T$ followed by another with $\mathbf{V}'$, which is highly efficient on GPU tensor cores. However, since $\gamma_i = \prod_{j=1}^i \mathbf{f}_j$ and each element in $\mathbf{f}_j$ is in the range of $(0, 1)$, the value in γ_i can be very small when i is large. Thus, the value in $\mathbf{V}'$ might explode, causing numerical instability. This means that materializing $\mathbf{V}'$ and then using existing kernel for standard softmax attention is infeasible. Therefore, *FiX* needs a more numerically stable implementation.

3.2.2. FALLBACK TO PARALLEL FORM

Due to the numerical instability in the matrix form, we have to resort to the parallel form as in the first line of Equation 13. Nonetheless, while the parallel form eliminates the need for division, it cannot leverage the full parallel computing power of GPU Tensor Cores. Specifically, for each query vector $\mathbf{q}_i$, the corresponding value vectors $\mathbf{v}_j$ from previous steps are not the same. Taking $\mathbf{v}_1$ as an example, in *FiX*, $\mathbf{q}_1$ uses $\mathbf{v}_1$ but $\mathbf{q}_2$ requires $\mathbf{v}_1 \circ \mathbf{f}_2$, and so on. Thus, we can not combine all the transformed values into one common $\mathbf{V}$ matrix. Instead, the calculation must be done separately for each query position. This results in many independent *matrix-vector* multiplication operations, rather than a single *matrix-matrix* multiplication operation.

3.2.3. RESCALE TRICK

Though it seems that the numerical stability and matrix multiplication are conflict to each other, we can use a rescale trick to balance them.

Suppose $m \leq i$, then

$$\begin{aligned} \mathbf{o}_i &= \frac{\left[\sum_{j=1}^m \exp(\mathbf{q}_i \mathbf{k}_j^T) \mathbf{v}_j \circ \frac{\gamma_m}{\gamma_j} \right]}{\sum_{j=1}^i \exp(\mathbf{q}_i \mathbf{k}_j^T)} \circ \frac{\gamma_i}{\gamma_m} \\ &\quad + \frac{\sum_{j=m+1}^i \exp(\mathbf{q}_i \mathbf{k}_j^T) \mathbf{v}_j \circ \frac{\gamma_i}{\gamma_j}}{\sum_{j=1}^i \exp(\mathbf{q}_i \mathbf{k}_j^T)}. \end{aligned} \quad (18)$$

The above equation avoids division by small numbers. In the first line, the condition $j \leq m \leq i$ ensures that the ratios γ_m/γ_j and γ_i/γ_m are less than 1. Furthermore, for a given m and any query $\mathbf{q}_i$ where $i \geq m$, the term $\mathbf{v}_j \circ \frac{\gamma_m}{\gamma_j}$ is constant. This property allows the summation

in the first line of Equation 18 to be computed as a matrix multiplication.

The FlashAttention (Dao, 2024) algorithm naturally splits Q and O into chunks in its outer loop. For each chunk, we select the largest m such that $m \leq i$ for any index i in that chunk. This approach allows most of the computation to be performed using matrix multiplication while maintaining numerical stability. The details of the efficient implementation for Equation 18 are discussed in Section 4.

3.3. Connection to Related Work

Relations with FoX. It is evident that VO-FoX is a special case of FiX when f_i falls back to a scalar forget gate. Meanwhile, as previously discussed in Section 2.4, the VO-FoX is equivalent to the standard FoX (QK-FoX) under RMSNorm. Thus, FiX can be viewed as a generalization of FoX, which fixes the lack of fine-grained forget gate.

Relations with positional encoding on QK. Existing QK positional encoding falls mainly into two categories (Zhang et al., 2025a): *additive* and *multiplicative*. Specifically, additive position encoding, such as ALiBi (Press et al., 2022) and FoX (Lin et al., 2025), adds a bias term to the attention logit, i.e., $q_i k_j^T + D_{i,j}$, where $D_{i,j}$ is a position-dependent bias. Multiplicative position encoding, such as RoPE (Su et al., 2024) and PaTH (Yang et al., 2025c), applies a transformation between q_i and k_j^T , i.e., $q_i T_{i,j} k_j^T$, where $T_{i,j}$ is a position-dependent transformation matrix. FiX is compatible with them as they are all applied to q and k while FiX is applied to v and o . For example, combined with RoPE, all q , k , v , and o receive appropriate position encoding:

$$o_i = \frac{\sum_{j=1}^i \exp(q_i R_{i-j} k_j^T) (v_j \circ \prod_{t=j+1}^i f_t)}{\sum_{j=1}^i \exp(q_i R_{i-j} k_j^T)}, \quad (19)$$

where R_{i-j} is the RoPE rotation for relative position $i - j$.

4. Efficient and Stable Implementation

In this part, we first introduce our Flash Fine-grained Attention implementation in Section 4.1, and then discuss how we fuse the RMSNorm into the attention and linear kernels for numerical stability in Section 4.2. Finally, we explain how to reduce the memory consumption of the extra F cache during inference in Section 4.3.

4.1. Flash Fine-grained Attention

Standard softmax attention, while highly parallelizable, generates intermediate matrices of size $s \times s$, where s is the sequence length. The $O(s^2)$ memory requirement for these matrices creates a bottleneck in memory access and makes long context training impractical. To overcome this, FlashAttention (Dao, 2024) avoids materializing the full

attention matrix in High-Bandwidth Memory (HBM) by computing the attention output using online softmax algorithm on SRAM. This is a memory-efficient approach that has become the standard of attention implementations.

However, as discussed in Section 3.2, we cannot directly use the standard FlashAttention due to numerical stability issues. We therefore propose *Flash Fine-grained Attention*, an adaptation that maintains efficiency while ensuring stability.

Our method follows the tiling strategy of FlashAttention. For simplicity, we assume the query Q , key K , value V , and output O matrices are all partitioned into n blocks of size B . An outer loop processes blocks of Q and O in parallel across different thread blocks on the GPU. Within each thread block, an inner loop iterates over the blocks of K and V . For a given query block Q_i , the first time step in Q_i is $(i - 1) \times B + 1$, and we set it to the m in Equation 18. Then, when iterating through key/value blocks K_j , V_j , we distinguish two cases for attention computation:

- If $j < i$, all time steps in V_j are smaller than m . The computation for this block corresponds to the first term in Equation 18 with standard matrix multiplication.
- If $j = i$, the block V_j contains time steps larger than m . Thus, the computation corresponds to the second term in Equation 18, where we compute each o_i individually.

The number of blocks where $j < i$ scales quadratically ($O(s^2)$) with the sequence length, whereas the number of diagonal blocks where $j = i$ scales linearly ($O(s)$). Consequently, the vast majority of the computation can be calculated as matrix multiplication and accelerated using highly optimized GPU Tensor Cores. We implement Flash Fine-grained Attention for both forward and backward passes, with detailed algorithms provided in Appendix E.

4.2. Fused RMSNorm and Linear Layer

The Root Mean Square Layer Normalization (RMSNorm) is defined as follows. Given an input vector $x \in \mathbb{R}^{d_v}$ and a weight $w \in \mathbb{R}^{d_v}$, the output $y \in \mathbb{R}^{d_v}$ is computed by:

$$s = \sqrt{\frac{1}{d_v} \sum_{i=1}^{d_v} x_i^2 + \epsilon}, \quad (20)$$

$$y = \frac{x}{s} \circ w.$$

The scaling factor s is in the denominator, which can lead to numerical instability if s is close to zero. A small constant ϵ , typically 10^{-5} or 10^{-6} , is added to prevent this instability.

In our work, we set ϵ to a much smaller value of 10^{-30} , which increases the risk of instability. During training, intermediate results are often downcast to `bfloat16` format before being written to HBM from SRAM to conserve mem-

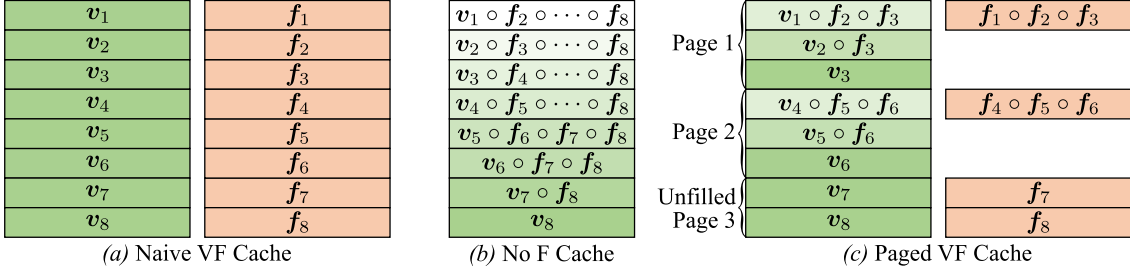


Figure 2. Comparison of cache strategies for managing the forget gate (F) and value (V) cache.

ory bandwidth. With our small ϵ , the reduced precision of bfloat16 is insufficient to preserve numerical stability during the division, and thus this computation must be performed in the float32 precision format.

Since our attention kernel holds the output in on-chip SRAM using float32 format before the output is finally written back to HBM, we can fuse the RMSNorm forward pass within the Flash Fine-grained Attention kernel. This allows the normalization to be applied in high precision, avoiding the HBM round-trip and associated precision loss. The similar challenge also arises in the backward pass, because the gradient with respect to x , $\frac{\partial \mathcal{L}}{\partial x}$, also involves a division by s , as shown in the following formula:

$$c = \sum_{i=1}^{d_o} y_i \frac{\partial \mathcal{L}}{\partial y_i}, \quad (21)$$

$$\frac{\partial \mathcal{L}}{\partial x} = \frac{1}{s} \left(\frac{\partial \mathcal{L}}{\partial y} \circ w - c \frac{y}{w} \right).$$

Note that the gradient $\frac{\partial \mathcal{L}}{\partial y}$ is also read from HBM in bfloat16 format. Therefore, the division by s still causes numerical instability. As the $\frac{\partial \mathcal{L}}{\partial y}$ is the output of the backward pass of the subsequent linear layer, to conduct the division in high precision, we fuse the RMSNorm’s backward pass within the linear layer’s backward pass to perform the calculation using the on-chip gradients in float32 format.

Combining these forward and backward requirements, we fuse the attention, RMSNorm, and the subsequent linear layer into a single FiX-RMSNorm-Linear operator. This single fused operator ensures the numerical stability throughout the training process. Details of the implementation for the forward and backward passes are provided in Algorithm 3 and Algorithm 4 in Appendix E, respectively.

4.3. Paged VF Cache for Inference

Standard auto-regressive inference with attention requires a KV cache to store past keys and values. By introducing a fine-grained forgetting gate, FiX additionally requires an element-wise forget gate, which necessitates an F cache. However, we find that we can mitigate this extra storage with minimal computation overheads by absorbing the F

cache into V cache with paged management. Note that as FiX makes no changes to the key structures, we only focus on the V and F caches in the following discussion.

As shown in Figure 2a, the naive approach is to store the V and F caches separately. However, the forget gate f must be stored in float32 for numerical stability, while keys and values are typically stored in bfloat16. Consequently, this naive F cache doubles the memory requirement by requiring an extra overhead equal to the standard KV cache. An alternative, as depicted in Figure 2b, absorbs the cumulative forget gates directly into the V cache at each step and eliminate the additional F Cache. However, this approach has two major limitations: (1) it doubles the memory bandwidth required for the V Cache, as every v must be read and re-written at each decoding step to incorporate a new f ; (2) the repeated conversions from bfloat16 to float32 for multiplication and back to bfloat16 for storage cause cumulative quantization errors that degrade value precision.

To balance the memory footprint and numerical precision, we propose *paged VF cache*. Specifically, we manage the V and F caches in fixed-size pages and absorb F cache into V cache within each page. For example, as illustrated in Figure 2c, there are 8 tokens and we set the page size p to 3. For tokens not filling a complete page (namely token 7 and 8), their generated V and F values are stored in a new, unfilled page separately in their own caches. Once a page is completely filled (e.g., page 1 and 2), the cumulative forget gates within that page are absorbed into the values. Absorbed V caches in complete pages are frozen. Besides, we only store the product of the forget gates as the F cache, e.g., $f_1 \circ f_2 \circ f_3$. For subsequent steps, this product is used to scale the values in V cache before that page. For example, $f_4 \circ f_5 \circ f_6$ is used to scale the V caches in page 1.

This paged VF cache offers two benefits: (1) it brings minimal memory overhead to store the products of F values in float32, which is roughly $1/p$ of the standard KV cache; (2) it also avoids updating the entire V cache at every step and only requires to update the v vector in a page every p decoded tokens. In practice, we find that setting p to 16 achieves a good balance between computation and memory. Furthermore, this paged management can be seamlessly in-

tegrated into popular inference engines like vLLM (Kwon et al., 2023) and SGLang (Zheng et al., 2024b), since they already use PagedAttention (Kwon et al., 2023). We implement this cache update mechanism, and the algorithm is detailed in Appendix D.

5. Experiments

We experiment with *FiX* and compare it against two important position encoding baselines: RoPE (Su et al., 2024) and FoX (Lin et al., 2025). Because *FiX* and FoX can both work with RoPE, we also experiment with *FiX*-RoPE and FoX-RoPE. We train all the models on a subset of FineWeb-Edu (Lozhkov et al., 2024), a collection of educational text filtered from FineWeb. Detailed architectural specifications are provided in Appendix B.

5.1. Training Setup

All models were trained using the AdamW (Loshchilov & Hutter, 2019) optimizer with $\beta = (0.9, 0.95)$ and a cosine learning rate schedule. We employed a warmup period of 512M tokens, where the learning rate increased linearly from 0 to a peak of 1×10^{-3} before decaying to a final value of 1×10^{-4} . We applied a weight decay of 0.1, a global gradient clipping of 1.0, and a batch size of 512K tokens. Models with approximately 760M parameters are trained on 48B tokens (92160 steps), while 340M models are trained on 10B tokens (20480 steps). Most parameters were initialized with a standard deviation of 0.02, with the exception of specific forget-gate related parameters.

The initialization of b_f and $\log a$ in Equation 14 follows Mamba (Gu & Dao, 2024), GDN (Yang et al., 2025b), and KDA (Zhang et al., 2025b). The parameters are initialized as the following equation:

$$\begin{aligned} A &\sim \text{Uniform}(a_{\min}, a_{\max}), \\ D_i &\sim \text{Uniform}(\log \Delta_{\min}, \log \Delta_{\max}), \\ \log a &= \log A, \\ \Delta_i &= \max(\exp(D_i), \Delta_{\text{floor}}), \\ b_{f,i} &= \text{softplus}^{-1}(\Delta_i), \end{aligned} \quad (22)$$

where $\text{softplus}(x) = \log(1 + e^x)$.

In this paper, we set $a_{\min} = 0$, $a_{\max} = 16$, $\Delta_{\min} = 10^{-3}$, $\Delta_{\max} = 10^{-1}$, and $\Delta_{\text{floor}} = 10^{-4}$.

5.2. Training Loss

We first compare the training loss of different models on identical training data. Due to stochastic fluctuations across batches, the raw loss curves can be noisy and obscure the subtle yet consistent differences between models. To provide a clearer comparison, we report the loss difference of each model relative to *FiX*.

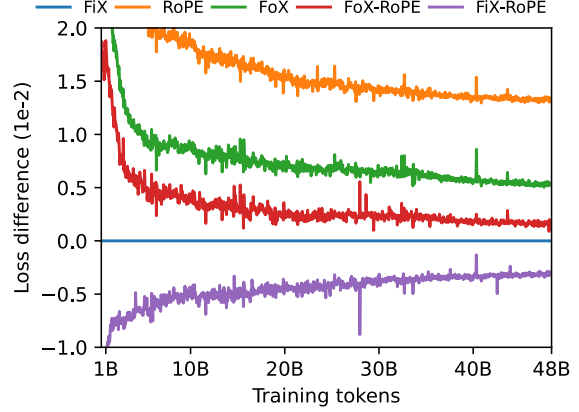


Figure 3. Training loss difference relative to *FiX*, calculated as $\mathcal{L}_{\text{model}} - \mathcal{L}_{\text{FiX}}$. Models with positive values (above the zero line) are outperformed by *FiX*.

Figure 3 plots these differences, calculated as $\mathcal{L}_{\text{model}} - \mathcal{L}_{\text{FiX}}$. A positive value (above the zero line) indicates that the model’s loss is higher than *FiX*’s, while a negative value signifies better performance than *FiX*. As shown in the figure, RoPE, FoX, and their combination FoX-RoPE all exhibit consistently positive loss differences, indicating that they underperform *FiX* throughout the whole training. In contrast, *FiX*-RoPE the only model that consistently remains in the negative region, achieving a lower loss than standalone *FiX*. These results suggests two key insights: (1) *FiX* is the most effective standalone position encoding method among the ones tested; (2) *FiX* can be effectively combined with RoPE to achieve further performance gains, highlighting its compatibility and synergistic potential.

5.3. Common Sense Benchmarks

We evaluate the models on a range of zero-shot common sense reasoning tasks. Specifically, we report the model perplexity on Wikitext (Merity et al., 2016) and LAMBADA (Kazemi et al., 2023), as well as accuracy on LAMBADA, PIQA (Bisk et al., 2020), HellaSwag (Zellers et al., 2019), WinoGrande (Sakaguchi et al., 2020), ARC-easy, ARC-challenge (Clark et al., 2018), SIQA (Sap et al., 2019), and BoolQ (Clark et al., 2019).

The results in Table 1 highlight the strong and consistent performance of *FiX*. Among all models, *FiX* and *FiX*-RoPE achieve the highest average and geometric mean accuracies, demonstrating their superior overall capability. Notably, standalone *FiX* achieves the best average accuracy (55.24) and geometric mean (53.79), yielding the top or second-best result in 7 out of the 10 reported metrics. This consistent performance gain across the diverse set of benchmarks underscores its reliability and robust generalization ability. We notice that *FiX* achieves slightly better results than *FiX*-RoPE despite that *FiX*-RoPE has lower training loss. This is

Table 1. Zero-shot evaluation on common sense reasoning benchmarks and language modeling perplexity. We report perplexity (ppl) on Wikitext (Wiki.) and LAMBADA (LMB.), and accuracy (acc/acc_n) for eight downstream tasks. The average (Avg.) and geometric mean (Geo Mean.) are calculated over the accuracies of these eight tasks. Best results are in **bold**, and second-best are underlined.

Model	Wiki. ppl ↓	LMB. ppl ↓	LMB. acc ↑	PIQA acc ↑	Hella. acc_n ↑	Wino. acc ↑	ARC-e acc ↑	ARC-c acc_n ↑	SIQA acc ↑	BoolQ acc ↑	Avg. ↑	Geo Mean. ↑
RoPE	18.21	13.39	46.90	71.33	55.12	59.35	67.63	<u>36.26</u>	41.30	<u>60.64</u>	54.82	53.51
FoX	17.88	13.35	45.82	71.55	55.83	59.12	68.81	34.98	<u>40.69</u>	58.56	54.42	52.98
FoX-RoPE	17.66	13.14	46.23	<u>72.20</u>	56.05	60.54	70.58	36.52	39.97	55.26	54.67	53.23
FiX	<u>17.72</u>	<u>12.69</u>	<u>47.78</u>	72.58	56.32	<u>60.06</u>	68.35	35.32	40.63	60.86	55.24	53.79
FiX-RoPE	17.85	12.32	48.55	70.89	<u>56.25</u>	59.35	<u>69.23</u>	36.09	40.58	57.80	<u>54.85</u>	<u>53.52</u>

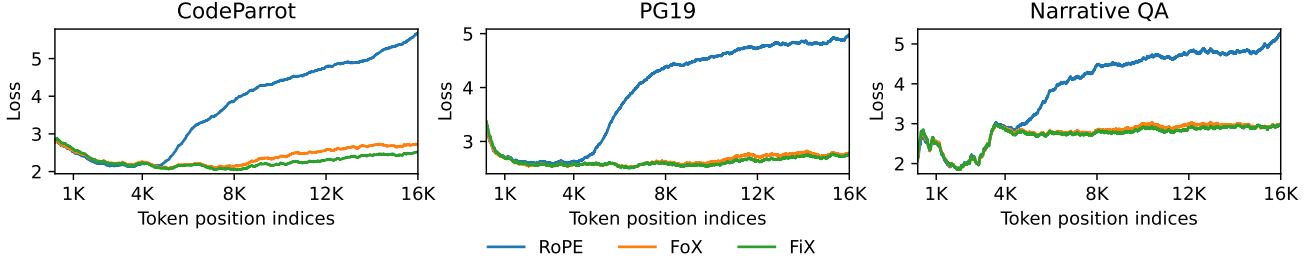


Figure 4. Length extrapolation performance on CodeParrot, PG-19, and NarrativeQA. Models were trained with a context length of 4096 and evaluated on sequences up to 16,384. The metric is the average loss at each token position.

because the pretraining loss of a model does not necessarily reflect its performance on certain downstream tasks.

5.4. Long Context Tasks

Length extrapolation. In Figure 4, we evaluate the models’ ability to handle contexts longer than those seen during training, without any fine-tuning. We compare *FiX* to RoPE, FoX on three long-context datasets: PG-19 (books) (Rae et al., 2020), CodeParrot (code), and NarrativeQA (dialogue) (Kočíský et al., 2018). Following FoX (Lin et al., 2025), we use per-token loss to sketch the ability of length extrapolation. We train all models with a context length of 4096 and tested them on sequences up to 16,384, measuring the average loss at each token position. The results show that both FoX and *FiX* maintain stable and low loss up to 16,384 tokens. Notably, *FiX* consistently performs slightly better than FoX across all three datasets, demonstrating its superior long-context modeling capabilities. In contrast, RoPE’s performance collapses sharply when the context length exceeds the 4096-token training limit, suggesting its inability to extrapolate. As this catastrophic failure would dominate the results, we do not combine RoPE with *FiX* and FoX in this experiment.

BABILong. To further assess the long-context reasoning ability, we evaluate all models on the BABILong (Kuratov et al., 2024) benchmark, which tests a model’s ability to reason about facts distributed across extremely long documents. Figure 5 reports the average accuracy across BABILong’s five question-answering (QA) tasks as context length in-

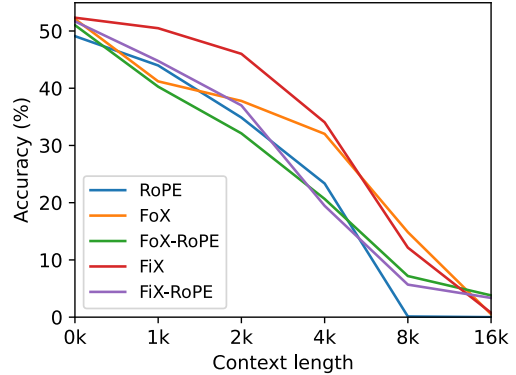


Figure 5. Average accuracy on the BABILong benchmark across varying context lengths. *FiX* demonstrates superior performance, especially in the 1k-4k context range.

creases. The results show that *FiX* consistently demonstrates a good model accuracy, significantly outperforming other baselines, particularly in the 1k-4k context length range. We also decompose the results on 5 different types of QA questions in BABILong, detailed in Appendix C.1. We observe that *FiX* outperforms baselines on most of the types, further validating its effectiveness.

5.5. Ablation Study

We conduct an ablation study to justify three key design choices in *FiX*: (1) setting $\epsilon = 10^{-30}$ in the output RMSNorm, (2) using a forget gate embedding (Equation 15) in the first layer, and (3) employing a Mamba-style acti-

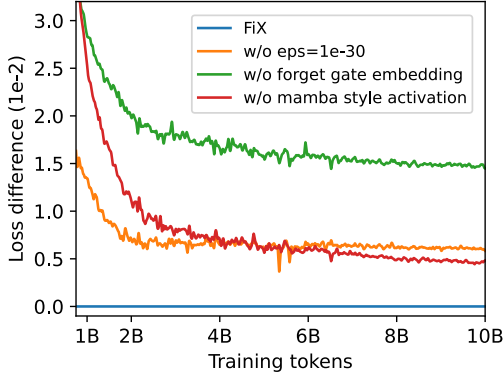


Figure 6. Ablation study of *FiX*’s design choices. The plot shows the training loss difference relative to the full *FiX* model ($\mathcal{L}_{\text{ablated}} - \mathcal{L}_{\text{FiX}}$). All ablated variants perform worse, confirming the importance of each component.

vation (Equation 14) for the forget gate. We create the three model variants by reverting each of these designs from our proposed *FiX* architecture, specifically: (1) replacing $\epsilon = 10^{-30}$ with a standard value of $\epsilon = 10^{-6}$, (2) using a low-rank projection for the forget gate in the first layer instead of an embedding, (3) using a simple sigmoid activation for the forget gate.

We report the training loss difference of these ablated models relative to the complete *FiX* model in Figure 6. The results show that all three variants have positive loss differences, indicating that their performance is worse than that of the full *FiX* model. This confirms that all three design choices are crucial to achieve the optimal performance.

We further support the necessity of these components through theoretical analysis. In particular, the small $\epsilon = 10^{-30}$ is essential for maintaining the equivalence between VO-FoX and QK-FoX. The forget gate embedding in the first layer is required to stabilize training, as a standard projection layer at this stage suffers from an exploding gradient norm. Finally, while the Mamba-style activation enhances performance, it is only effective for fine-grained gates, not scalar ones. This suggests that the primary performance gain stems from our novel fine-grained forget gate.

Equivalence of VO-FoX and QK-FoX. Figure 7 shows the loss difference compared to FoX. When ϵ is set to 10^{-30} , the loss difference of VO-FoX and FoX converges to 0. This proves the equivalence of VO-FoX and QK-FoX when ϵ is close to 0. In contrast, when ϵ is set to 10^{-6} , the VO-FoX is outperformed by FoX, which again shows the importance of setting ϵ to 10^{-30} .

The effect of Mamba-style activation function. In Figure 7, we keep $\epsilon = 10^{-30}$ in VO-FoX and change the activation function of the forget gate from simple sigmoid function to Mamba-style activation function. Different with the result presented in Figure 6, the Mamba-style activation

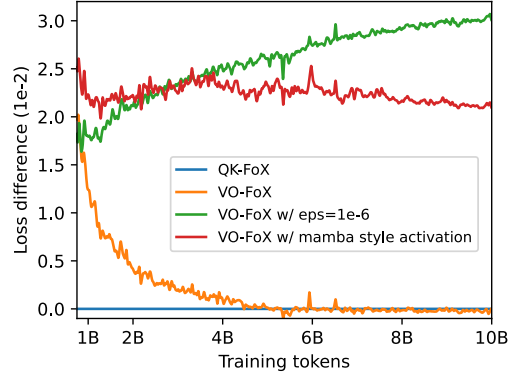


Figure 7. Study of the equivalence of VO-FoX and QK-FoX. The plot shows the training loss difference relative to the FoX model ($\mathcal{L}_{\text{ablated}} - \mathcal{L}_{\text{FoX}}$).

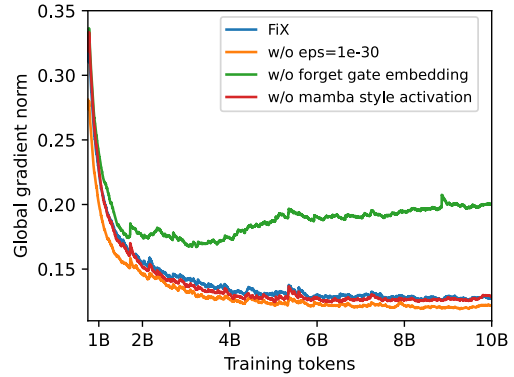


Figure 8. Ablation study of *FiX*’s design choices. The plot shows the global gradient norm against the training steps.

function performs worse than sigmoid function when only using a scalar forget gate. This verifies that the performance gain of *FiX* originates from the our fine-grained forget gate rather than Mamba-style activation function, even though it is necessary for *FiX*.

6. Conclusion

This paper introduces *FiX*, which incorporates a fine-grained forget gate into softmax attention by applying it to the v and o vectors. Notably, *FiX* is the first positional encoding on V-O vectors as far as we understand. For efficient and numerically stable training and inference, we implement fused Flash Fine-grained Attention and a paged VF cache. We compare *FiX* against RoPE and FoX on various tasks and experimental results demonstrate that *FiX* outperforms these baselines both in terms of training loss and model accuracy. We also show that *FiX* can be seamlessly combined with RoPE to produce further improvements.

Acknowledgments

This work is partially supported by the National Natural Science Foundation of China (Grant Nos. 62422206, 62532007, 62532001) and the Hong Kong Research Grants Council under the Theme-based Research Scheme (project no. T41-517/25-N). This work is also a research gift from AlayaDB.AI Inc.

Impact Statement

This work is focused on advancing the architecture of neural networks. By proposing a method to create more powerful transformers, our research contributes to the general progress in artificial intelligence. While this work is primarily aimed to accelerate positive applications in science, education, and accessibility, we acknowledge that more capable models also inherit the risks associated with large language models, such as the potential generation of sophisticated misinformation. We advocate for the continued development of safety protocols and responsible deployment practices in parallel with such performance improvements.

References

- Bisk, Y., Zellers, R., Le bras, R., Gao, J., and Choi, Y. PIQA: Reasoning about physical commonsense in natural language. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):7432–7439, Apr. 2020.
- Chi, T.-C., Fan, T.-H., Ramadge, P., and Rudnicky, A. KER-PL: Kernelized relative positional embedding for length extrapolation. In *Advances in Neural Information Processing Systems*, 2022.
- Clark, C., Lee, K., Chang, M.-W., Kwiatkowski, T., Collins, M., and Toutanova, K. BoolQ: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 2924–2936, Minneapolis, Minnesota, June 2019. Association for Computational Linguistics.
- Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C., and Tafjord, O. Think you have solved question answering? try ARC, the AI2 reasoning challenge, 2018.
- Csordás, R., Irie, K., and Schmidhuber, J. The neural data router: Adaptive control flow in transformers improves systematic generalization. In *International Conference on Learning Representations*, 2022.
- Dao, T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *The Twelfth International Conference on Learning Representations*, 2024.
- DeepSeek-AI, Liu, A., Feng, B., Xue, B., Wang, B., Wu, B., Lu, C., Zhao, C., Deng, C., Zhang, C., Ruan, C., Dai, D., Guo, D., et al. DeepSeek-v3 technical report, 2025.
- Dehghani, M., Djolonga, J., Mustafa, B., Padlewski, P., Heek, J., Gilmer, J., Steiner, A. P., Caron, M., Geirhos, R., Alabdulmohsin, I., Jenatton, R., Beyer, L., et al. Scaling vision transformers to 22 billion parameters. In *Proceedings of the 40th International Conference on Machine Learning*, volume 202, pp. 7480–7512. PMLR, 23–29 Jul 2023.
- Grattafiori, A., Dubey, A., Jauhri, A., Pandey, A., Kadian, A., Al-Dahle, A., Letman, A., Mathur, A., Schelten, A., Vaughan, A., Yang, A., et al. The Llama 3 herd of models, 2024.
- Gu, A. and Dao, T. Mamba: Linear-time sequence modeling with selective state spaces, 2024.
- Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, November 1997.
- Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are RNNs: fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*. JMLR.org, 2020.
- Kazemi, M., Kim, N., Bhatia, D., Xu, X., and Ramachandran, D. LAMBADA: Backward chaining for automated reasoning in natural language. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 6547–6568, Toronto, Canada, July 2023. Association for Computational Linguistics.
- Kočiský, T., Schwarz, J., Blunsom, P., Dyer, C., Hermann, K. M., Melis, G., and Grefenstette, E. The NarrativeQA reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 6:317–328, 2018.
- Kogkalidis, K., Bernardy, J.-P., and Garg, V. Algebraic positional encodings. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- Kuratov, Y., Bulatov, A., Anokhin, P., Rodkin, I., Sorokin, D. I., Sorokin, A., and Burtsev, M. BABILong: Testing the limits of LLMs with long context reasoning-in-a-haystack. In *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2024.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J., Zhang, H., and Stoica, I. Efficient memory management for large language model serving

- p>with pagedattention. In
- Proceedings of the 29th Symposium on Operating Systems Principles*
- , pp. 611–626, New York, NY, USA, 2023. Association for Computing Machinery.
- Leviathan, Y., Kalman, M., and Matias, Y. Selective attention improves transformer. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Li, S., You, C., Guruganesh, G., Ainslie, J., Ontanon, S., Zaheer, M., Sanghai, S., Yang, Y., Kumar, S., and Bhojanapalli, S. Functional interpolation for relative positions improves long context transformers. In *The Twelfth International Conference on Learning Representations*, 2024.
- Lin, Z., Nikishin, E., He, X. O., and Courville, A. C. Forgetting transformer: Softmax attention with a forget gate. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025.
- Loshchilov, I. and Hutter, F. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.
- Lozhkov, A., Ben Allal, L., von Werra, L., and Wolf, T. Fineweb-Edu: the finest collection of educational content, 2024.
- Merity, S., Xiong, C., Bradbury, J., and Socher, R. Pointer sentinel mixture models, 2016.
- Press, O., Smith, N., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *International Conference on Learning Representations*, 2022.
- Rae, J. W., Potapenko, A., Jayakumar, S. M., Hillier, C., and Lillicrap, T. P. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations*, 2020.
- Sakaguchi, K., Le Bras, R., Bhagavatula, C., and Choi, Y. WinoGrande: An adversarial winograd schema challenge at scale. *Proceedings of the AAAI Conference on Artificial Intelligence*, 34(05):8732–8740, Apr. 2020.
- Sap, M., Rashkin, H., Chen, D., Le Bras, R., and Choi, Y. Social IQa: Commonsense reasoning about social interactions. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 4463–4473, Hong Kong, China, November 2019. Association for Computational Linguistics.
- Shazeer, N. Glu variants improve transformer, 2020.
- Su, J. The evolution of transformers 19: Rotary position encoding of the second type, Apr 2025.
- Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., and Liu, Y. RoFormer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. ISSN 0925-2312.
- Sun, Y., Dong, L., Huang, S., Ma, S., Xia, Y., Xue, J., Wang, J., and Wei, F. Retentive network: A successor to transformer for large language models. *CoRR*, abs/2307.08621, 2023.
- Tan, S., Yang, S., Courville, A., Panda, R., and Shen, Y. Scaling stick-breaking attention: An efficient implementation and in-depth study. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Team, ., Zeng, A., Lv, X., Zheng, Q., Hou, Z., Chen, B., Xie, C., Wang, C., Yin, D., Zeng, H., Zhang, J., Wang, K., Zhong, L., Liu, M., Lu, R., Cao, S., et al. GLM-4.5: Agentic, reasoning, and coding (ARC) foundation models, 2025.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems 2017, December 4-9, 2017, Long Beach, CA, USA*, pp. 5998–6008, 2017.
- Veisi, A., Amirzadeh, H., and Mansourian, A. Context-aware biases for length extrapolation, 2025.
- Wen, K., Dang, X., and Lyu, K. RNNs are not transformers (yet): The key bottleneck on in-context retrieval. In *The Thirteenth International Conference on Learning Representations*, 2025.
- Yang, A., Li, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Gao, C., Huang, C., Lv, C., Zheng, C., Liu, D., Zhou, F., Huang, F., Hu, F., Ge, H., Wei, H., Lin, H., Tang, J., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Zhou, J., Lin, J., Dang, K., Bao, K., Yang, K., Yu, L., Deng, L., Li, M., Xue, M., Li, M., Zhang, P., Wang, P., Zhu, Q., Men, R., Gao, R., Liu, S., Luo, S., Li, T., Tang, T., Yin, W., Ren, X., Wang, X., Zhang, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Zhang, Y., Wan, Y., Liu, Y., Wang, Z., Cui, Z., Zhang, Z., Zhou, Z., and Qiu, Z. Qwen3 technical report, 2025a.
- Yang, S., Wang, B., Shen, Y., Panda, R., and Kim, Y. Gated linear attention transformers with hardware-efficient training. In *Forty-first International Conference on Machine Learning, ICML 2024, Vienna, Austria, July 21-27, 2024*. OpenReview.net, 2024.

- Yang, S., Kautz, J., and Hatamizadeh, A. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations, ICLR 2025, Singapore, April 24-28, 2025*. OpenReview.net, 2025b.
- Yang, S., Shen, Y., Wen, K., Tan, S., Mishra, M., Ren, L., Panda, R., and Kim, Y. PaTH attention: Position encoding via accumulating householder transformations. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025c.
- Zellers, R., Holtzman, A., Bisk, Y., Farhadi, A., and Choi, Y. HellaSwag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*, pp. 4791–4800, Florence, Italy, July 2019. Association for Computational Linguistics.
- Zhang, B. and Sennrich, R. Root mean square layer normalization. In *Advances in Neural Information Processing Systems*, volume 32. Curran Associates, Inc., 2019.
- Zhang, Y., Chen, Z., Liu, Y., Qin, Z., Yuan, H., Xu, K., Yuan, Y., Gu, Q., and Yao, A. C.-C. Group representational position encoding, 2025a.
- Zhang, Y., Lin, Z., Yao, X., Hu, J., Meng, F., Liu, C., Men, X., Yang, S., Li, Z., Li, W., Lu, E., Liu, W., Chen, Y., Xu, W., Yu, L., Wang, Y., Fan, Y., Zhong, L., Yuan, E., Zhang, D., Zhang, Y., Liu, T. Y., Wang, H., Fang, S., He, W., Liu, S., Li, Y., Su, J., Qiu, J., Pang, B., Yan, J., Jiang, Z., Huang, W., Yin, B., You, J., Wei, C., Wang, Z., Hong, C., Chen, Y., Chen, G., Wang, Y., Zheng, H., Wang, F., Liu, Y., Dong, M., Zhang, Z., Pan, S., Wu, W., Wu, Y., Guan, L., Tao, J., Fu, G., Xu, X., Wang, Y., Lai, G., Wu, Y., Zhou, X., Yang, Z., and Du, Y. Kimi Linear: An expressive, efficient attention architecture. *CoRR*, abs/2510.26692, 2025b.
- Zheng, C., Gao, Y., Shi, H., Huang, M., Li, J., Xiong, J., Ren, X., Ng, M., Jiang, X., Li, Z., and Li, Y. DAPE: Data-adaptive positional encoding for length extrapolation. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024a.
- Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024b.

A. Related Work

Existing positional encodings in softmax attention can be broadly classified into two categories: additive and multiplicative, as detailed in Section 3.3. Multiplicative methods construct a matrix T_{ij} to modulate the query-key interaction, forming $q_i T_{ij} k_j^T$. Early approaches such as RoPE (Su et al., 2024) and APE (Kogkalidis et al., 2024) employ data-independent constructions for T_{ij} . In contrast, more recent work like PaTH (Yang et al., 2025c) introduces data-dependent transformations using Householder-like matrices. Additive methods, on the other hand, directly modify the attention logits. Early works in this category, including ALiBi (Press et al., 2022), Kerple (Chi et al., 2022), and FIRE (Li et al., 2024), add a data-independent bias $b_{ij} = f(i, j)$ to the logits $q_i k_j^T$. Subsequent methods like DAPE (Zheng et al., 2024a), FoX (Lin et al., 2025), CABLE (Veisi et al., 2025), and Selective Attention (Leviathan et al., 2025) incorporate data-dependent biases of the form $b_{ij} = f(i, j, x_j, \dots, x_i)$. GRAPE (Zhang et al., 2025a) provides a unifying perspective on these two categories through the lens of group theory. Beyond these, alternative mechanisms such as Stick-breaking Attention (Tan et al., 2025) (a causal variant of Geometric Attention (Csordás et al., 2022)) replace the softmax function with a stick-breaking process to generate attention scores.

All these methods focus on the attention logits or scores, effectively making them positional encoding for the query-key (QK) component. While some approaches, such as RoPE, can be mathematically adapted for the value-output (VO) computation as demonstrated in VO-RoPE (Su, 2025), their performance in this context is inferior to the original QK application. In fact, combining VO-RoPE with QK-RoPE often yields worse results than using QK-RoPE alone. In contrast, *FiX* integrates positional information into the VO computation by introducing a fine-grained forget gate, which makes *FiX* the first state-of-the-art positional encoding for v and o vectors.

B. Architectures

All attention variants were tested under a modern and consistent transformer architecture (equipped with SwiGLU (Shazeer, 2020), QK-Norm (Dehghani et al., 2023), O-Norm, and Short-Conv (Gu & Dao, 2024)). This ensures that any performance differences can be directly attributed to the attention mechanisms. The models used in the primary experiments have approximately 760M parameters, whereas those used for ablation studies have approximately 340M parameters. The 760M-parameter model has 36 hidden layers, 10 heads, and a SwiGLU intermediate size of 3840. The 340M-parameter model has 24 hidden layers, 8 heads, and a SwiGLU intermediate size of 3072. The bottleneck dimension of the low-rank forget gate projection in *FiX* is 128. Thus, the low-rank projection only introduces approximately 8-10M parameters.

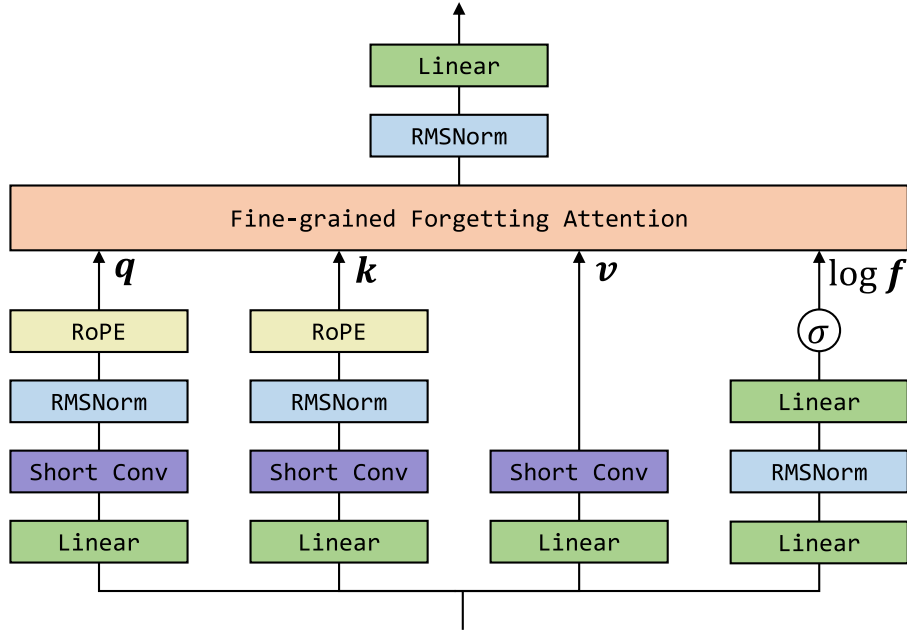


Figure 9. The architecture of the attention layer in *FiX* combined with RoPE

Figure 9 illustrates the model architecture of the attention layer when our proposed *FiX* is used in conjunction with Rotary

Position Embeddings (RoPE). The activation function, denoted by σ in the diagram, is described in Equation 14. This architecture is adapted for the different models in our experiments.

For models that do not employ RoPE, the corresponding module is simply omitted. When implementing the FoX model, the ‘‘Fine-grained Forgetting Attention’’ is replaced by ‘‘Forgetting Attention’’, wherein $\log f$ is a scalar generated by a single linear layer with a simple sigmoid activation. For the model that only uses RoPE, the attention layer reverts to a standard causal softmax attention mechanism.

Apart from these specified modifications, all models evaluated in our experiments share this common architectural framework to ensure a fair comparison.

C. Experiments Continued

C.1. BABILong

Figure 10 presents the accuracy of different models across individual QA tasks in BABILong.

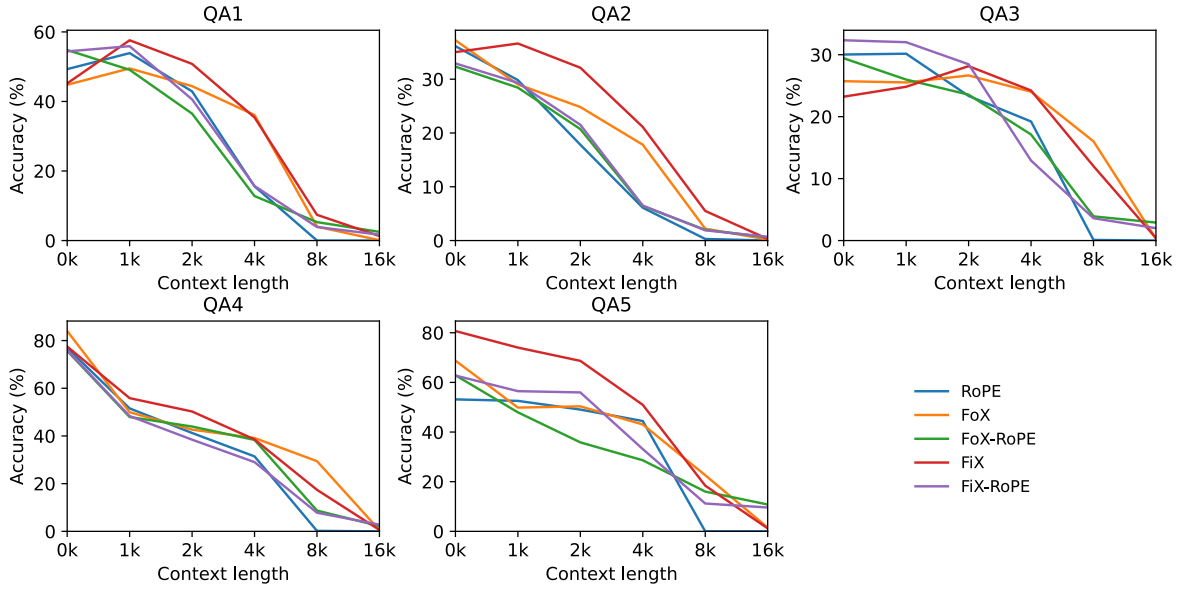


Figure 10. Accuracy on individual BABILong tasks across varying context lengths.

Gradient Norm. Figure 8 presents the global gradient norms of different models vs. the training steps. The global gradient norm is defined as $\sqrt{\sum_{i=1}^n x_i^2}$, where n is the total number of parameters of a model, and x_i is the i -th parameter. It shows that if we do not adopt the forget gate embedding, the gradient norm will be unbounded, because it is hard to learn a meaningful projection layer from the discrete token embeddings.

D. Update Algorithm of Paged VF Cache

We present the algorithm of VF cache update during the prefilling stage in Algorithm 1 and decoding stage in Algorithm 2.

Algorithm 1 VF Cache Update of FiX during Prefilling Stage

- 1: **Global State:**
- 2: $\mathbf{K}_{\text{cache}}, \mathbf{V}_{\text{cache}}$: Key/Value Paged Cache tensors.
- 3: $\mathbf{S}_{\text{cache}}$: Cache for sum of $\log f$ for each full page.
- 4: $\mathbf{F}_{\text{current}}$: Cache for $\log \mathbf{F}$ values of the last, unfilled page.
- 5: $\mathbf{M}[s, p]$: Mapping from sample s , page index p to a global page ID.
- 6: $\mathbf{N}[s]$: Number of pages for sample s .
- 7: $\mathbf{L}[s]$: Current length for sample s .

```

8:  $i$ : The first available page index.
9:  $B$ : Page size as a constant.
10: Input: Batch of sequences  $\mathbf{K}, \mathbf{V}, \log \mathbf{F}$  and number of samples  $n$ .
11: for  $s = 0$  to  $n - 1$  do
12:   Get the input of sample  $s$ :  $\mathbf{K}^{(s)}, \mathbf{V}^{(s)}, \log \mathbf{F}^{(s)}$  from  $\mathbf{K}, \mathbf{V}, \log \mathbf{F}$ 
13:   Get the length  $l_s$  of sample  $s$ .
14:    $L[s] \leftarrow l_s$ .
15:    $p_s \leftarrow \lceil \frac{l_s}{B} \rceil$  is the number of pages for sample  $s$ .
16:    $N[s] \leftarrow p_s$ .
17:   for  $p = 0$  to  $p_s - 1$  do
18:     Get the input slice for page  $p$ :  $\mathbf{K}_p^{(s)}, \mathbf{V}_p^{(s)}, \log \mathbf{F}_p^{(s)}$  from  $\mathbf{K}^{(s)}, \mathbf{V}^{(s)}, \log \mathbf{F}^{(s)}$ .
19:      $i_p \leftarrow i$  is the page index for  $p$ .
20:      $M[s, p] \leftarrow i_p$ .
21:      $i \leftarrow i + 1$ .
22:      $\mathbf{K}_{\text{cache}}[i_p] \leftarrow \mathbf{K}_p^{(s)}$ .
23:     if  $p < p_s - 1$  then
24:        $\log \mathbf{\Gamma}_p^{(s)} \leftarrow \text{CumulativeSum}(\log \mathbf{F}_p^{(s)})$ .
25:        $\mathbf{S}_p \leftarrow \log \mathbf{\Gamma}_p^{(s)}[-1]$ .
26:        $\mathbf{V}_{\text{cache}}[i_p] \leftarrow \mathbf{V}_p^{(s)} \odot \exp(\mathbf{S}_p - \log \mathbf{\Gamma}_p^{(s)})$ .
27:        $\mathbf{S}_{\text{cache}}[i_p] \leftarrow \mathbf{S}_p$ .
28:     else
29:        $\mathbf{V}_{\text{cache}}[i_p] \leftarrow \mathbf{V}_p^{(s)}$ .
30:        $\mathbf{F}_{\text{current}}[s] \leftarrow \log \mathbf{F}_p^{(s)}$ .
31:     end if
32:   end for
33: end for
    
```

Algorithm 2 VF Cache Update of *FiX* during Decoding Stage

```

1: Global State:
2:  $\mathbf{K}_{\text{cache}}, \mathbf{V}_{\text{cache}}$ : Key/Value Paged Cache tensors.
3:  $\mathbf{S}_{\text{cache}}$ : Cache for sum of  $\log \mathbf{F}$  for each full page.
4:  $\mathbf{F}_{\text{current}}$ : Cache for  $\log \mathbf{F}$  values of the last, unfilled page.
5:  $M[s, p]$ : Mapping from sample  $s$ , page index  $p$  to a global page ID.
6:  $N[s]$ : Number of pages for sample  $s$ .
7:  $L[s]$ : Current length for sample  $s$ .
8:  $i$ : The first available page index.
9:  $B$ : Page size as a constant.
10: Input: New tokens  $\mathbf{k}_n, \mathbf{v}_n, \log \mathbf{f}_n$  and number of samples  $n$ .
11: for  $s = 0$  to  $n - 1$  do
12:   Get the current length  $l_s \leftarrow L[s]$ .
13:   if  $l_s > 0$  and  $l_s \equiv 0 \pmod{B}$  then
14:      $p_{\text{prev}} \leftarrow l_s/B - 1$ .
15:      $i_p \leftarrow M[s, p_{\text{prev}}]$ .
16:      $\log \mathbf{F}_{\text{page}} \leftarrow \mathbf{F}_{\text{current}}[s]$ .
17:      $\mathbf{V}_{\text{page}} \leftarrow \mathbf{V}_{\text{cache}}[i_p]$ .
18:      $\log \mathbf{\Gamma}_{\text{page}} \leftarrow \text{CumulativeSum}(\log \mathbf{F}_{\text{page}})$ .
19:      $\mathbf{S}_{\text{page}} \leftarrow \log \mathbf{\Gamma}_{\text{page}}[-1]$ .
20:      $\mathbf{V}_{\text{cache}}[i_p] \leftarrow \mathbf{V}_{\text{page}} \odot \exp(\mathbf{S}_{\text{page}} - \log \mathbf{\Gamma}_{\text{page}})$ .
21:      $\mathbf{S}_{\text{cache}}[i_p] \leftarrow \mathbf{S}_{\text{page}}$ .
22:   end if
23:   Get the new token for sample  $s$ :  $\mathbf{k}_n^{(s)}, \mathbf{v}_n^{(s)}, \log \mathbf{f}_n^{(s)}$  from  $\mathbf{k}_n, \mathbf{v}_n, \log \mathbf{f}_n$ .
    
```

```

24:  $p \leftarrow \lfloor l_s / B \rfloor$  is the page index for the new token.
25:  $j \leftarrow l_s \bmod B$  is the index within the page.
26: if  $j = 0$  then
27:      $i_p \leftarrow i$ .
28:      $M[s, p] \leftarrow i_p$ .
29:      $N[s] \leftarrow p + 1$ .
30:      $i \leftarrow i + 1$ .
31: else
32:      $i_p \leftarrow M[s, p]$ .
33: end if
34:  $K_{\text{cache}}[i_p, j] \leftarrow k_n^{(s)}$ .
35:  $V_{\text{cache}}[i_p, j] \leftarrow v_n^{(s)}$ .
36:  $F_{\text{current}}[s, j] \leftarrow \log f_n^{(s)}$ .
37:  $L[s] \leftarrow l_s + 1$ .
38: end for
    
```

E. Flash Fine-grained Attention

We present the implementation of Flash Fine-grained Attention in this section using runnable pseudo-code. The whole implementation includes 4 kernels and a function that integrates these kernels.

Algorithm 3 presents the forward pass of Flash Fine-grained Attention.

Algorithm 3 The Forward Pass of Flash Fine-grained Attention

```

def flash_fine_grained_attention_forward(
    q: Tensor,          # [h, s, d_k], bfloat16
    k: Tensor,          # [h, s, d_k], bfloat16
    v: Tensor,          # [h, s, d_v], bfloat16
    log_g: Tensor,      # [h, s, d_v], float32
    rmsnorm_weight: Tensor, # [d_v], bfloat16
    sm_scale: float,
    eps: float,
    BM: int = 64,
    BN: int = 64,
):
    # dimensions
    h, s, d_k = q.shape
    d_v = v.shape[-1]

    # final output
    m = empty(h, s, dtype= float32)
    l = empty(h, s, dtype= float32)
    rms = empty(h, s, dtype= float32)
    o = empty(h, s, d_v, dtype=bfloat16)

    # outer loop, executed in parallel across different thread blocks
    for i in range((s + BM - 1) // BM):
        # load q and log_g for block 'i' from HBM, and calculate the indices of q and o
        real_bm = BM if (i + 1) * BM < s else s - i * BM
        indices_m = i * BM + arange(0, BM)

        bq = zeros(h, BM, d_k, dtype=bfloat16)
        b_log_g_m = zeros(h, BM, d_v, dtype= float32)

        bq[:, :, real_bm].copy_(q[:, :, i * BM: i * BM + real_bm])
        b_log_g_m[:, :, real_bm].copy_(log_g[:, :, i * BM: i * BM + real_bm])

        # variables used in online softmax:
        # row-wise maximum, denominator of softmax, output
        bm = full([h, BM], -inf, dtype=float32)
        bl = zeros([h, BM], dtype=float32)
        bo = zeros([h, BM, d_v], dtype=float32)
    
```

```

# inner loop
# 'i * BM' is the number of tokens before the first token of q in block 'i'
# '(i * BM + BN - 1) // BN' is ceil(i * BM / BN)
for j in range((i * BM + BN - 1) // BN):
    # load k, v, and log_g for block 'j' from HBM, and calculate the indices of k and v
    real_bn = BN if (j + 1) * BN < s else s - j * BN
    indices_n = j * BN + arange(0, BN)

    bk = zeros(h, BN, d_k, dtype=bfloat16)
    bv = zeros(h, BN, d_v, dtype=bfloat16)
    b_log_g_n = zeros(h, BN, d_v, dtype=float32)

    bk[:, :real_bn].copy_(k[:, j * BN: j * BN + real_bn])
    bv[:, :real_bn].copy_(v[:, j * BN: j * BN + real_bn])
    b_log_g_n[:, :real_bn].copy_(log_g[:, j * BN: j * BN + real_bn])

    # calculate the numerator of softmax(qk.T + mask) for block 'j'
    # only the tokens before the first token of q in block 'i' can be attended
    bqkT = bmm(bq, bk.transpose(1, 2), float32) * sm_scale
    mask = (indices_m[:, None] < s) & (indices_n[None, :] < i * BM)
    bqkT = where(mask, bqkT, -inf)
    b_new_m = maximum(bm, amax(bqkT, dim=-1))
    scores = exp(bqkT - b_new_m.view(h, BM, 1))

    # naively, this should be 'bv * exp(-b_log_g_n)'
    # however, this is numerically unstable, because the scale can be very large
    # instead, we use 'exp(b_log_g_m[:, 1] - b_log_g_n)'
    # because the index of b_log_g_m[:, 0] is i * BM and 'indices_n[None, :] < i * BM',
    # all valid log_v_scale <= 0, there is no risk of dividing v by a near-zero value
    log_v_scale = minimum(b_log_g_m[:, 1] - b_log_g_n, tensor(0.))
    bv = (bv.to(float32) * exp(log_v_scale)).to(bfloat16)

    # update bm, bl, bo according to online softmax algorithm
    bo *= exp(bm - b_new_m).view(h, BM, 1)
    bl *= exp(bm - b_new_m)
    bo += bmm(scores.to(bv.dtype), bv, float32)
    bl += sum(scores, dim=-1)
    bm = b_new_m

# naively, this should be 'bo * exp(b_log_g_m)'
# since the 'b_log_g_m[0]' is already used by all o vectors
# we use 'b_log_g_m - b_log_g_m[0]'
bo = bo * exp(b_log_g_m - b_log_g_m[:, :1])

bk = zeros(h, BM, d_k, dtype=bfloat16)
bv = zeros(h, BM, d_v, dtype=bfloat16)
bk[:, :real_bm].copy_(k[:, i * BM: i * BM + real_bm])
bv[:, :real_bm].copy_(v[:, i * BM: i * BM + real_bm])

bqkT = bmm(bq, bk.transpose(1, 2), float32) * sm_scale
mask = (indices_m[:, None] < s) & (indices_m[None, :] <= indices_m[:, None])
bqkT = where(mask, bqkT, -inf)
b_new_m = maximum(bm, amax(bqkT, dim=-1))
scores = exp(bqkT - b_new_m.view(h, BM, 1))

bo *= exp(bm - b_new_m).view(h, BM, 1)
bl *= exp(bm - b_new_m)
# because no intermediate log_g can be found, we resort to parallel form
# bo1 = s11 * (v1 / g1) * g1
# bo2 = s21 * (v1 / g1) * g2 + s22 * (v2 / g2) * g2
# bo3 = s31 * (v1 / g1) * g3 + s32 * (v2 / g2) * g3 + s33 * (v3 / g3) * g3
# ...
# thus, bo = s[:, 1] * v1 * (g / g1) + s[:, 2] * v2 * (g / g2) + ...
for j in range(BM):
    scale = exp(minimum(b_log_g_m - b_log_g_m[:, j:j+1], tensor(0.)))
    bo += scores[:, :, j:j+1] * scale * bv[:, j:j+1].to(float32)
bl += sum(scores, dim=-1)
bm = b_new_m

# normalized by the denominator of the softmax
bo = bo / bl.view(h, BM, 1)

# applying rms norm
b_rms = sqrt(mean(bo**2, dim=-1) + eps)
bo = (bo * rmsnorm_weight.to(float32)) / b_rms.view(h, BM, 1)
    
```

```

# cast to bfloat16 before writing back to HBM
bo = bo.to(bfloat16)

m[:,i * BM: i * BM + real_bm].copy_(bm[:,real_bm])
l[:,i * BM: i * BM + real_bm].copy_(bl[:,real_bm])
rms[:,i * BM: i * BM + real_bm].copy_(b_rms[:,real_bm])
o[:,i * BM: i * BM + real_bm].copy_(bo[:,real_bm])

return m, l, rms, o

```

Algorithm 4 presents one of the backward pass of Flash Fine-grained Attention. It calculates the backward pass of fused linear layer and RMSNorm and other statistics that is necessary for later usage.

Algorithm 4 The Backward Pass of the Fused Linear Layer

```

def flash_fine_grained_attention_backward_linear(
    normed_o: Tensor,      # [h, s, d_v], bfloat16
    d_final_o: Tensor,     # [s, d_h], bfloat16
    linear_weight: Tensor, # [d_h, h * d_v], bfloat16
    rmsnorm_weight: Tensor, # [d_v], bfloat16
    rms: Tensor,           # [h, s], float32
):
    h, s, d_v = normed_o.shape
    rms_w = rmsnorm_weight.to(float32)

    do = empty(h, s, d_v, dtype=bfloat16)
    d_weight = zeros(d_v, dtype=float32)
    sum_sds = empty(h, s, dtype=float32)
    d_log_g = empty(h, s, d_v, dtype=float32)

    # different heads are processes in parallel across different thread blocks
    for i_h in range(h):
        # backward of linear layer
        b_linear_weight = linear_weight[:,i_h * d_v: (i_h + 1) * d_v]
        bd_normed_o = mm(d_final_o, b_linear_weight, float32)

        # backward of rmsnorm: input
        b_normed_o = normed_o[i_h].to(float32)
        rms_s = rms[i_h].view(-1, 1)
        ydy = b_normed_o * bd_normed_o
        c = mean(ydy, dim=-1, keepdim=True)
        bdo = (bd_normed_o * rms_w - (c * b_normed_o) / rms_w) / rms_s
        do[i_h] = bdo.to(bfloat16)

        # backward of rmsnorm: weight
        bd_weight = sum(ydy / rms_w, dim=0)
        d_weight += bd_weight

        # backward of fine-grained attn:
        # row-wise sum of softmax(qk.T + mask) * d(softmax(qk.T + mask))
        # used later when calculating the gradient of q and k
        odo = ydy - (c * b_normed_o**2) / (rms_w**2)
        b_sum_sds = sum(odo, dim=-1)
        sum_sds[i_h] = b_sum_sds

        # backward of fine-grained attn: log_g through o
        bd_log_g = odo
        d_log_g[i_h] = bd_log_g

    d_weight = d_weight.to(bfloat16)
    return do, d_weight, sum_sds, d_log_g

```

Algorithm 5 presents one of the backward pass of Flash Fine-grained Attention. It calculates the gradient with respect to Q .

Algorithm 5 The Backward Pass of Flash Fine-grained Attention: Q

```

def flash_fine_grained_attention_backward_q(
    q: Tensor,      # [h, s, d_k], bfloat16
    k: Tensor,      # [h, s, d_k], bfloat16

```

```

v: Tensor,      # [h, s, d_v], bfloat16
log_g: Tensor,  # [h, s, d_v], bfloat16
do: Tensor,     # [h, s, d_v], bfloat16
sum_sds: Tensor, # [h, s],      bfloat16
m: Tensor,      # [h, s],      bfloat16
l: Tensor,      # [h, s],      bfloat16
sm_scale: float,
BM: int = 64,
BN: int = 64,
):
    # dimensions
    h, s, d_k = q.shape
    d_v = v.shape[-1]

    # final output
    dq = empty(h, s, d_k, dtype=bfloat16)

    # outer loop, executed in parallel across different thread blocks
    for i in range((s + BM - 1) // BM):
        real_bm = BM if (i + 1) * BM < s else s - i * BM
        indices_m = i * BM + arange(0, BM)

        bdo = zeros(h, BM, d_v, dtype=bfloat16)
        bq = zeros(h, BM, d_k, dtype=bfloat16)
        b_log_g_m = zeros(h, BM, d_v, dtype= float32)
        b_sum_sds = zeros(h, BM, dtype= float32)
        bm = zeros(h, BM, dtype= float32)
        bl = zeros(h, BM, dtype= float32)

        bdo[:, :real_bm].copy_(do[:, i * BM: i * BM + real_bm])
        bq[:, :real_bm].copy_(q[:, i * BM: i * BM + real_bm])
        b_log_g_m[:, :real_bm].copy_(log_g[:, i * BM: i * BM + real_bm])
        b_sum_sds[:, :real_bm].copy_(sum_sds[:, i * BM: i * BM + real_bm])
        bm[:, :real_bm].copy_(m[:, i * BM: i * BM + real_bm])
        bl[:, :real_bm].copy_(l[:, i * BM: i * BM + real_bm])

        b_sum_sds = b_sum_sds.view(h, BM, 1)
        bm = bm.view(h, BM, 1)
        bl = bl.view(h, BM, 1)

        bdo_scaled = (bdo.to(float32) * exp(b_log_g_m - b_log_g_m[:, :1])).to(bfloat16)

        # accumulator of dq for block 'i'
        bdq = zeros([h, BM, d_k], dtype=float32)

        # non-diagonal blocks
        for j in range((i * BM + BN - 1) // BN):
            real_bn = BN if (j + 1) * BN < s else s - j * BN
            indices_n = j * BN + arange(0, BN)

            bk = zeros(h, BN, d_k, dtype=bfloat16)
            bv = zeros(h, BN, d_v, dtype=bfloat16)
            b_log_g_n = zeros(h, BN, d_v, dtype= float32)

            bk[:, :real_bn].copy_(k[:, j * BN: j * BN + real_bn])
            bv[:, :real_bn].copy_(v[:, j * BN: j * BN + real_bn])
            b_log_g_n[:, :real_bn].copy_(log_g[:, j * BN: j * BN + real_bn])

            # recompute s = softmax(qk.T + mask)
            bqkT = bmm(bq, bk.transpose(1, 2), float32) * sm_scale
            mask = (indices_m[:, None] < s) & (indices_n[None, :] < i * BM)
            bqkT = where(mask, bqkT, -inf)
            bs = exp(bqkT - bm) / bl

            # compute ds
            bv_scaled = (bv.to(float32) * exp(b_log_g_m[:, :1] - b_log_g_n)).to(bfloat16)
            bds = bmm(bdo_scaled, bv_scaled.transpose(1, 2), float32)

            # compute dq
            bdqkT = (bs * (bds - b_sum_sds)) * sm_scale
            bdq += bmm(bdqkT.to(bfloat16), bk, float32)

        # diagonal block
        bk = zeros(h, BM, d_k, dtype=bfloat16)
        bv = zeros(h, BM, d_v, dtype=bfloat16)
        b_log_g_n = zeros(h, BM, d_v, dtype= float32)

```

```

bk     [:, :, real_bm].copy_(k[:, i * BM: i * BM + real_bm])
bv     [:, :, real_bm].copy_(v[:, i * BM: i * BM + real_bm])
b_log_g_n[:, :, real_bm].copy_(log_g[:, i * BM: i * BM + real_bm])

bqkT = bmm(bq, bk.transpose(1, 2), float32) * sm_scale
mask = (indices_m[:, None] < s) & (indices_m[None, :] <= indices_m[:, None])
bqkT = where(mask, bqkT, -inf)
bs = exp(bqkT - bm) / bl

# ds of the diagonal block can not be computed by matrix multiplication
bds = zeros(h, BM, BM, dtype=float32)
for j in range(BM):
    scale = exp(minimum(b_log_g_m - b_log_g_m[:, j:j+1], tensor(0.)))
    bds[:, :, j].copy_(sum(bdo.to(float32) * scale * bv[:, j:j+1].to(float32), dim=-1))

bdqkT = (bs * (bds - b_sum_sds)) * sm_scale
bdq += bmm(bdqkT.to(bfloat16), bk, float32)

# write results back to HBM
bdq = bdq.to(bfloat16)
dq[:, i * BM: i * BM + real_bm].copy_(bdq[:, :, real_bm])

return dq
    
```

Algorithm 6 presents one of the backward pass of Flash Fine-grained Attention. It calculates the gradient with respect to K , V , and $\log \Gamma$.

Algorithm 6 The Backward Pass of Flash Fine-grained Attention: K , V , and $\log \Gamma$

```

def flash_fine_grained_attention_backward_kv(
    q: Tensor,      # [h, s, d_k], bfloat16
    k: Tensor,      # [h, s, d_k], bfloat16
    v: Tensor,      # [h, s, d_v], bfloat16
    log_g: Tensor,  # [h, s, d_v], bfloat16
    do: Tensor,     # [h, s, d_v], bfloat16
    sum_sds: Tensor, # [h, s], bfloat16
    m: Tensor,      # [h, s], bfloat16
    l: Tensor,      # [h, s], bfloat16
    sm_scale: float,
    BM: int = 64,
    BN: int = 64,
):
    # dimensions
    h, s, d_k = q.shape
    d_v = v.shape[-1]

    # final outputs
    dk = empty(h, s, d_k, dtype=bfloat16)
    dv = empty(h, s, d_v, dtype=bfloat16)
    d_log_g = empty(h, s, d_v, dtype=float32)

    # outer loop, executed in parallel across different thread blocks
    for j in range((s + BN - 1) // BN):
        real_bn = BN if (j + 1) * BN < s else s - j * BN
        indices_n = j * BN + arange(0, BN)

        bk = zeros(h, BN, d_k, dtype=bfloat16)
        bv = zeros(h, BN, d_v, dtype=bfloat16)
        b_log_g_n = zeros(h, BN, d_v, dtype=float32)

        bk[:, :, real_bn].copy_(k[:, j * BN: j * BN + real_bn])
        bv[:, :, real_bn].copy_(v[:, j * BN: j * BN + real_bn])
        b_log_g_n[:, :, real_bn].copy_(log_g[:, j * BN: j * BN + real_bn])

        bv_scaled = (bv.to(float32) * exp(b_log_g_n[:, -1:] - b_log_g_n)).to(bfloat16)

        # accumulator of dk and dv for block 'j'
        bdk = zeros([h, BN, d_k], dtype=float32)
        bdv = zeros([h, BN, d_v], dtype=float32)

        # non-diagonal blocks
        # '(j + 1) * BN' is the index of the first token after block 'j'
    
```

```

# thus, '((j + 1) * BN) // BM' is the first non-diagonal block of q and o
begin = ((j + 1) * BN) // BM
end = (s + BM - 1) // BM
for i in range(min(begin, end), end):
    real_bm = BM if (i + 1) * BM < s else s - i * BM
    indices_m = i * BM + arange(0, BM)

    bdo = zeros(h, BM, d_v, dtype=bfloat16)
    bq = zeros(h, BM, d_k, dtype=bfloat16)
    b_log_g_m = zeros(h, BM, d_v, dtype=float32)
    b_sum_sds = zeros(h, BM, dtype=float32)
    bm = zeros(h, BM, dtype=float32)
    bl = ones(h, BM, dtype=float32)

    bdo[:, :real_bm].copy_(do[:, i * BM: i * BM + real_bm])
    bq[:, :real_bm].copy_(q[:, i * BM: i * BM + real_bm])
    b_log_g_m[:, :real_bm].copy_(log_g[:, i * BM: i * BM + real_bm])
    b_sum_sds[:, :real_bm].copy_(sum_sds[:, i * BM: i * BM + real_bm])
    bm[:, :real_bm].copy_(m[:, i * BM: i * BM + real_bm])
    bl[:, :real_bm].copy_(l[:, i * BM: i * BM + real_bm])

    b_sum_sds = b_sum_sds.view(h, BM, 1)
    bm = bm.view(h, BM, 1)
    bl = bl.view(h, BM, 1)

    # recompute s = softmax(qk.T + mask)
    bqkT = bmm(bq, bk.transpose(1, 2), float32) * sm_scale
    mask = ((indices_m < s) & (indices_m >= (j + 1) * BN))[:, None]
    bqkT = where(mask, bqkT, -inf)
    bs = exp(bqkT - bm) / bl

    # compute ds and dv
    bdo_scaled = (bdo.to(float32) * exp(b_log_g_m - b_log_g_n[:, -1])).to(bfloat16)
    bdv += bmm(bs.to(bfloat16).transpose(1, 2), bdo_scaled)
    bds = bmm(bdo_scaled, bv_scaled.transpose(1, 2), float32)

    # compute dk
    bdqkT = (bs * (bds - b_sum_sds)) * sm_scale
    bdk += bmm(bdqkT.to(bfloat16).transpose(1, 2), bq, float32)

bdv *= exp(b_log_g_n[:, -1] - b_log_g_n)

# diagonal block
bdo = zeros(h, BN, d_v, dtype=bfloat16)
bq = zeros(h, BN, d_k, dtype=bfloat16)
b_log_g_m = zeros(h, BN, d_v, dtype=float32)
b_sum_sds = zeros(h, BN, dtype=float32)
bm = zeros(h, BN, dtype=float32)
bl = ones(h, BN, dtype=float32)

bdo[:, :real_bn].copy_(do[:, j * BN: j * BN + real_bn])
bq[:, :real_bn].copy_(q[:, j * BN: j * BN + real_bn])
b_log_g_m[:, :real_bn].copy_(log_g[:, j * BN: j * BN + real_bn])
b_sum_sds[:, :real_bn].copy_(sum_sds[:, j * BN: j * BN + real_bn])
bm[:, :real_bn].copy_(m[:, j * BN: j * BN + real_bn])
bl[:, :real_bn].copy_(l[:, j * BN: j * BN + real_bn])

b_sum_sds = b_sum_sds.view(h, BN, 1)
bm = bm.view(h, BN, 1)
bl = bl.view(h, BN, 1)

bqkT = bmm(bq, bk.transpose(1, 2), float32) * sm_scale
mask = (indices_n[:, None] < s) & (indices_n[None, :] <= indices_n[:, None])
bqkT = where(mask, bqkT, -inf)
bs = exp(bqkT - bm) / bl

# ds and dv of the diagonal block can not be computed by matrix multiplication
bds = zeros(h, BN, BN, dtype=float32)
for i in range(BN):
    scale = exp(minimum(b_log_g_n[:, i:i+1] - b_log_g_n, tensor(0)))
    bdoi = bdo[:, i:i+1].to(float32)
    bdv += bs[:, i:i+1].transpose(1, 2) * scale * bdoi
    bds[:, i].copy_(sum(bdoi * scale * bv.to(float32), dim=-1))

bdqkT = (bs * (bds - b_sum_sds)) * sm_scale
bdk += bmm(bdqkT.to(bfloat16).transpose(1, 2), bq, float32)
    
```

```

# calculate d_log_g through v
bd_log_g = -bv.to(float32) * bdv

# write results back to HBM
bdk = bdk.to(bfloat16)
bdv = bdv.to(bfloat16)

dk   [:,j * BN:j * BN + real_bn].copy_(bdk[:,real_bn])
dv   [:,j * BN:j * BN + real_bn].copy_(bdv[:,real_bn])
d_log_g[:,j * BN:j * BN + real_bn].copy_(bd_log_g[:,real_bn])

return dk, dv, d_log_g

```

Algorithm 7 shows how to combine the previous kernels together to form the Flash Fine-grained Attention.

Algorithm 7 Flash Fine-grained Attention

```

class FusedFineGrainedAttentionNormLinear(Function):
    @staticmethod
    def forward(
        ctx: FunctionCtx,
        q: Tensor,          # [h, s, d_k], bfloat16
        k: Tensor,          # [h, s, d_k], bfloat16
        v: Tensor,          # [h, s, d_v], bfloat16
        log_g: Tensor,      # [h, s, d_v], float32
        rmsnorm_weight: Tensor, # [d_v], bfloat16
        linear_weight: Tensor, # [d_h, h * d_v], bfloat16
        sm_scale: float,
        eps: float,
    ):
        m, l, rms, normed_o = flash_fine_grained_attention_forward(
            q, k, v, log_g, rmsnorm_weight, sm_scale, eps)
        normed_o_input = rearrange(normed_o, 'h s d -> s (h d)')
        final_o = linear(normed_o_input, linear_weight)
        ctx.save_for_backward(
            q, k, v, log_g, rmsnorm_weight, linear_weight, m, l, rms, normed_o)
        ctx.sm_scale = sm_scale
        return final_o

    @staticmethod
    def backward(
        ctx: FunctionCtx,
        d_final_o: Tensor, # [s, d_h], bfloat16
    ):
        q, k, v, log_g, rms_w, linear_w, m, l, rms, normed_o = ctx.saved_tensors
        sm_scale = ctx.sm_scale
        normed_o_input = rearrange(normed_o, 'h s d -> s (h d)')
        d_linear_w = mm(d_final_o.T, normed_o_input, float32)
        do, d_rms_w, sum_sds, d_log_g_o = flash_fine_grained_attention_backward_linear(
            normed_o, d_final_o, linear_w, rms_w, rms)
        dq = flash_fine_grained_attention_backward_q(
            q, k, v, log_g, do, sum_sds, m, l, sm_scale)
        dk, dv, d_log_g_v = flash_fine_grained_attention_backward_kv(
            q, k, v, log_g, do, sum_sds, m, l, sm_scale)
        d_log_g = d_log_g_o + d_log_g_v
        return dq, dk, dv, d_log_g, d_rms_w, d_linear_w, None, None

```
